

UNIVERSITÉ EURO-MÉDITERRANÉENNE DE FÈS (UEMF)
ÉCOLE D'INGÉNIERIE DIGITALE ET D'INTELLIGENCE ARTIFICIELLE (EDIA)

WeatherOps

WeatherOps

DevSecOps Pipeline Report

Travaux Pratiques DevSecOps

Node.js · Express · Docker · Kubernetes
GitLab CI/CD · Security Scanning · Jest

Auteure

Asma Daoui

4^{ème} année – Cybersécurité

 github.com/BeeHope

Module

DevSecOps

Année académique 2024–2025

 Juin 2026

Abstract

Résumé du Projet

Ce rapport présente la conception et le déploiement de **WeatherOps**, une application web météo développée avec Node.js et Express, intégrée dans un pipeline DevSecOps complet. Le projet couvre l'ensemble du cycle de vie logiciel : développement, tests automatisés (Jest/Supertest), analyse de sécurité (détection de secrets), conteneurisation Docker, orchestration Kubernetes, et déploiement continu via GitLab CI/CD sur des environnements *staging* (branche **develop**) et *production* (branche **main**, déclenchement manuel).

Mots-clés : DevSecOps, Node.js, Express, Docker, Kubernetes, GitLab CI/CD, Jest, Supertest, Secret Detection, REST API.

Contents

Abstract	1
1 Introduction et Vue d'Ensemble	4
1.1 Contexte	4
1.2 Objectifs du TP	4
1.3 Architecture Globale	4
2 Application Node.js / Express	6
2.1 Description de l'Application	6
2.2 Endpoints REST	6
2.3 Structure du Code – <code>src/app.js</code>	6
2.4 Réponse JSON – Exemple	7
2.5 Captures d'Écran de l'Interface	7
3 Tests Automatisés	10
3.1 Framework de Test	10
3.2 Cas de Test – <code>tests/app.test.js</code>	10
3.3 Résultats des Tests	11
4 Conteneurisation Docker	12
4.1 Dockerfile et <code>.dockerignore</code>	12
4.2 Build et Test Local	12
4.3 Push vers le Registry GitLab	13
5 Infrastructure AWS et Configuration	14
5.1 Création de la Paire de Clés SSH	14
5.2 Instance EC2 de Staging	14
5.3 Déploiement via Terraform	14
6 Pipeline GitLab CI/CD	16
6.1 Configuration des Variables CI/CD	16
6.2 Vue d'Ensemble du Pipeline	16
6.3 Stage 1 – Test	17
6.4 Stage 2 – Security (Détection de Secrets)	17
6.5 Stage 3 – Build (Docker)	18
6.6 Stage 4 – Deploy Staging	18
6.7 Stage 5 – Deploy Production	19
6.8 Pipeline Réussi	19
7 Déploiement Kubernetes	21
7.1 Structure des Manifests	21

7.2	Vérification du Déploiement	21
8	Analyse de Sécurité	22
8.1	Vulnérabilités Identifiées	22
8.2	Bonnes Pratiques Appliquées	22
8.3	Améliorations Recommandées	22
9	Conclusion	23
A	Variables CI/CD GitLab	24
B	Données Météo – Référence	25

1. Introduction et Vue d'Ensemble

1.1 Contexte

WeatherOps est une application de démonstration développée dans le cadre du module *DevSecOps* de la 4^{ème} année d'ingénierie en cybersécurité à l'UEMF. Elle illustre les bonnes pratiques d'intégration de la sécurité (*Shift Left Security*) au sein d'un pipeline CI/CD moderne.

1.2 Objectifs du TP

- Développer une API REST météo avec Node.js et Express.
- Écrire et automatiser des tests unitaires avec Jest et Supertest.
- Implémenter une étape de sécurité (détection de secrets hardcodés).
- Conteneuriser l'application avec Docker.
- Déployer sur Kubernetes via GitLab CI/CD.
- Distinguer les environnements *staging* et *production*.

1.3 Architecture Globale

Architecture du Projet

```
weatherops/  
  src/  
    app.js           # Express app (routes + logique)  
    server.js        # Point d'entrée HTTP  
  views/             # Templates HTML (city, health, home, weather)  
    city.html  
    health.html  
    home.html  
    weather.html  
  tests/  
    app.test.js      # Tests Jest / Supertest  
  k8s/               # Manifests Kubernetes  
    namespace.yaml  
    deployment.yaml  
    service.yaml  
  Dockerfile  
  .gitlab-ci.yml  
  package.json
```

```
.env
```

2. Application Node.js / Express

2.1 Description de l'Application

WeatherOps expose une API météo simulée avec des données statiques pour 5 villes : Paris, London, Tokyo, New York, et Dubai. Chaque ville retourne des informations de conditions météorologiques (température, humidité, vitesse du vent, condition).

2.2 Endpoints REST

Méthode	Route	Description
GET	/	Dashboard principal — liste les villes
GET	/health	Statut de l'API (timestamp, status ok)
GET	/weather	Liste toutes les villes disponibles
GET	/weather/:city	Détails météo pour une ville

⚠ Vulnérabilité Détectée

Une clé API hardcodée a été identifiée dans `src/app.js` :

```
const API_KEY = "sk-demo-123456789"
```

Cette pratique est dangereuse. En production, les secrets doivent être injectés via des variables d'environnement ou un gestionnaire de secrets (HashiCorp Vault, AWS Secrets Manager). Le pipeline CI/CD inclut une étape de détection de ce type de secret.

2.3 Structure du Code – `src/app.js`

```
1 const express = require('express');
2 const path    = require('path');
3 const app     = express();
4
5 //      Ne jamais hardcoder de secrets en production
6 const API_KEY = "sk-demo-123456789";
7
8 app.use(express.json());
9 app.use(express.static(path.join(__dirname, '../public')));
10
11 // Données météo simulées
12 const weatherData = {
13   paris: { city: 'Paris',    temperature: 18,
14            condition: 'Partly Cloudy', humidity: 65,
15            wind_speed: 12, unit: 'C' },
16   london: { city: 'London',  temperature: 14,
```

```
17         condition: 'Rainy',          humidity: 80,  
18         wind_speed: 20, unit: 'C' },  
19     tokyo: { city: 'Tokyo',    temperature: 27,  
20             condition: 'Sunny',          humidity: 55,  
21             wind_speed: 8,  unit: 'C' },  
22     newyork: { city: 'New York', temperature: 22,  
23               condition: 'Clear',          humidity: 50,  
24               wind_speed: 15, unit: 'C' },  
25     dubai: { city: 'Dubai',    temperature: 40,  
26             condition: 'Hot & Sunny',    humidity: 30,  
27             wind_speed: 5,  unit: 'C' },  
28 };  
29  
30 module.exports = app;
```

Listing 2.1: Extrait de app.js – Structure principale

2.4 Réponse JSON – Exemple

GET /weather/paris

```
{  
  "city":      "Paris",  
  "country":   "France",  
  "temperature": "18C",  
  "condition": "Partly Cloudy",  
  "humidity":   65,  
  "wind_speed": 12,  
  "retrieved_at": "2026-06-03T10:00:00.000Z"  
}
```

2.5 Captures d'Écran de l'Interface

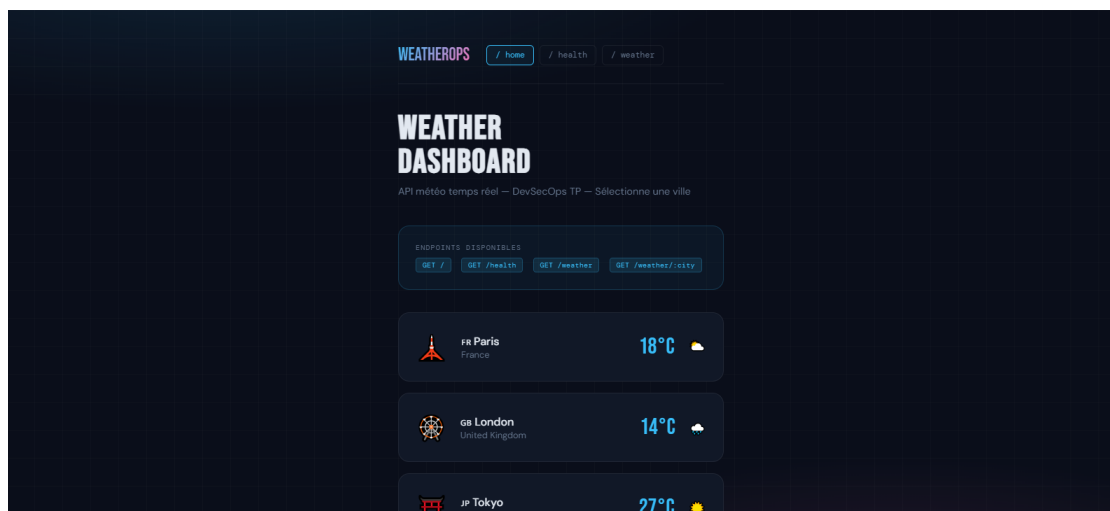


Figure 2.1: Page d'accueil WeatherOps – Dashboard principal (GET /)

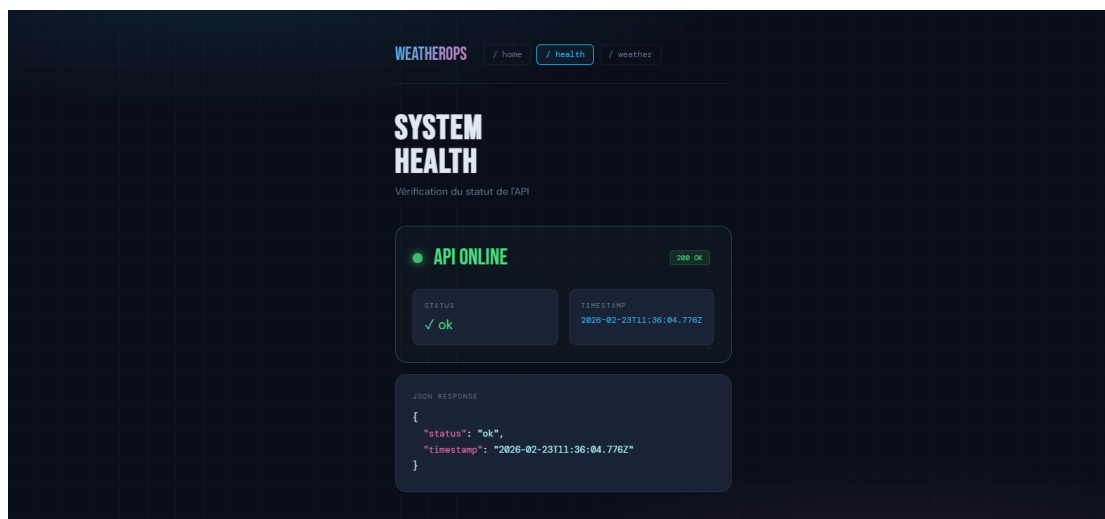


Figure 2.2: Page Health Check – API Online (GET /health)

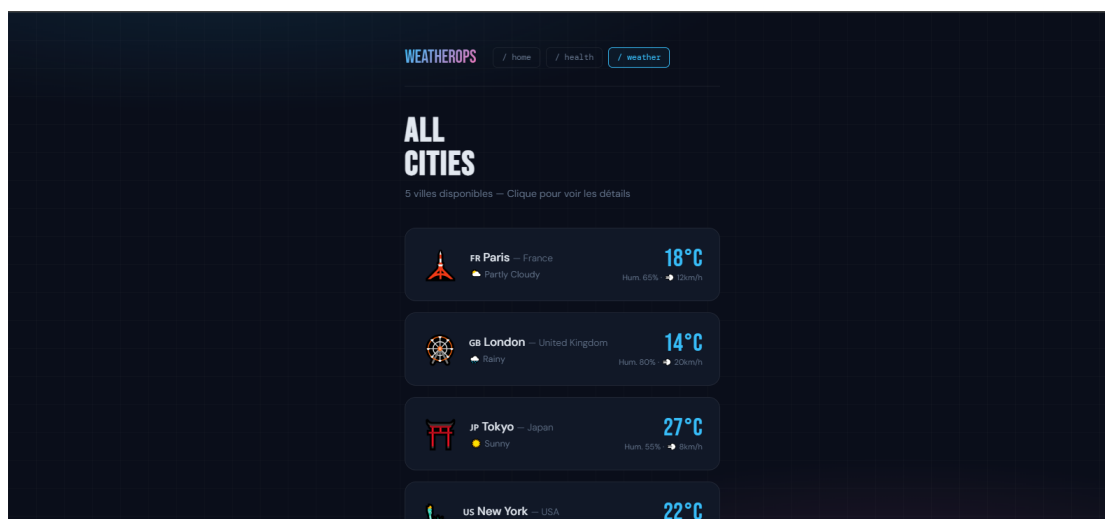


Figure 2.3: Page de liste des villes – All Cities (GET /weather)

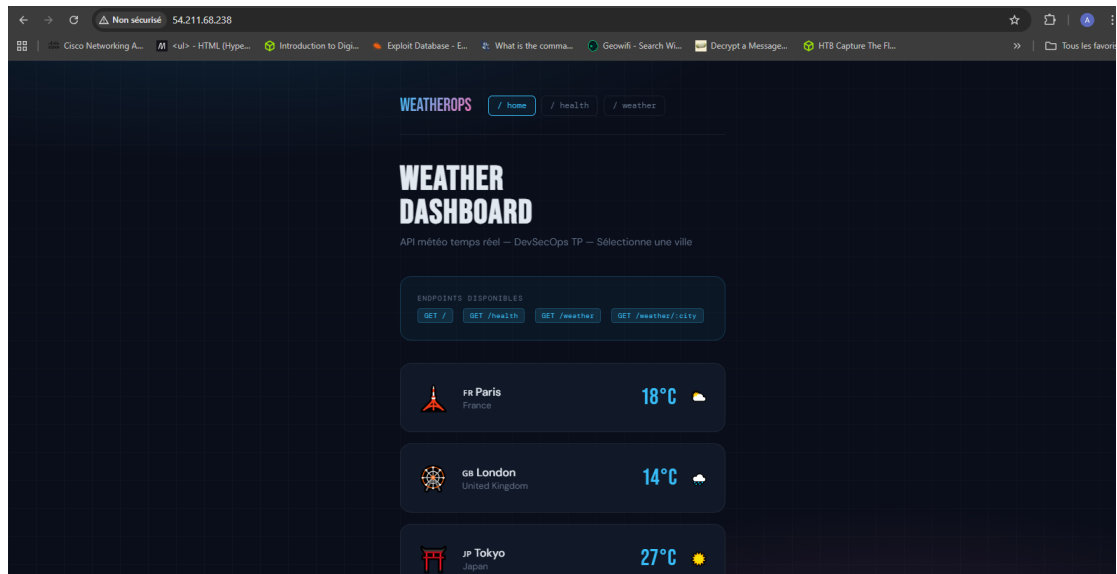


Figure 2.4: Application déployée et accessible via l'IP publique AWS (54.211.68.238)

3. Tests Automatisés

3.1 Framework de Test

Les tests sont écrits avec **Jest** (framework de test JavaScript) et **Supertest** (requêtes HTTP sans démarrer le serveur). Ces librairies sont déclarées en `devDependencies` dans `package.json`.

3.2 Cas de Test – `tests/app.test.js`

```
1  const request = require('supertest');
2  const app     = require('../src/app');
3
4  describe('Routes de base', () => {
5    test('GET / retourne du HTML', async () => {
6      const res = await request(app).get('/');
7      expect(res.status).toBe(200);
8      expect(res.text).toContain('<!DOCTYPE html>');
9      expect(res.text).toContain('DevSecOps');
10   });
11
12   test('GET /health retourne du HTML', async () => {
13     const res = await request(app).get('/health');
14     expect(res.status).toBe(200);
15     expect(res.text).toContain('API ONLINE');
16   });
17 });
18
19 describe('Routes Weather', () => {
20   test('GET /weather retourne du HTML', async () => {
21     const res = await request(app).get('/weather');
22     expect(res.status).toBe(200);
23     expect(res.text).toContain('Cities');
24   });
25
26   test('GET /weather/paris retourne du HTML', async () => {
27     const res = await request(app).get('/weather/paris');
28     expect(res.status).toBe(200);
29     expect(res.text).toContain('Paris');
30     expect(res.text).toContain('Partly Cloudy');
31   });
32
33   test('GET /weather/unknown retourne 404', async () => {
34     const res = await request(app).get('/weather/unknown');
35     expect(res.status).toBe(404);
36     expect(res.text).toContain('404');
37   });
38 });
```

Listing 3.1: `app.test.js` – Tests des routes

3.3 Résultats des Tests

✓ Résultats Jest

```
PASS tests/app.test.js
  Routes de base
    GET / retourne du HTML      (xx ms)
    GET /health retourne du HTML (xx ms)
  Routes Weather
    GET /weather retourne du HTML (xx ms)
    GET /weather/paris retourne du HTML (xx ms)
    GET /weather/tokyo retourne du HTML (xx ms)
    GET /weather/unknown retourne 404 (xx ms)

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
```

```
(node:3400) Warning: `--localstorage-file` was provided without a valid path
(Use `node --trace-warnings ...` to show where the warning was created)
PASS __tests__/app.test.js
  Routes de base
    ✓ GET / retourne du HTML (39 ms)
    ✓ GET /health retourne du HTML (8 ms)
  Routes Weather
    ✓ GET /weather retourne du HTML (11 ms)
    ✓ GET /weather/paris retourne du HTML (13 ms)
    ✓ GET /weather/tokyo retourne du HTML (12 ms)
    ✓ GET /weather/unknown retourne 404 (10 ms)

Test Suites: 1 passed, 1 total
Tests:       6 passed, 6 total
Snapshots:   0 total
Time:        1.38 s, estimated 2 s
Ran all test suites.
```

C:\Users\HP\Desktop\fourth year\Devsecops\tp-devsecops-app>

Figure 3.1: Résultats des tests Jest dans le terminal – 6 tests passés avec succès

4. Conteneurisation Docker

4.1 Dockerfile et .dockerignore

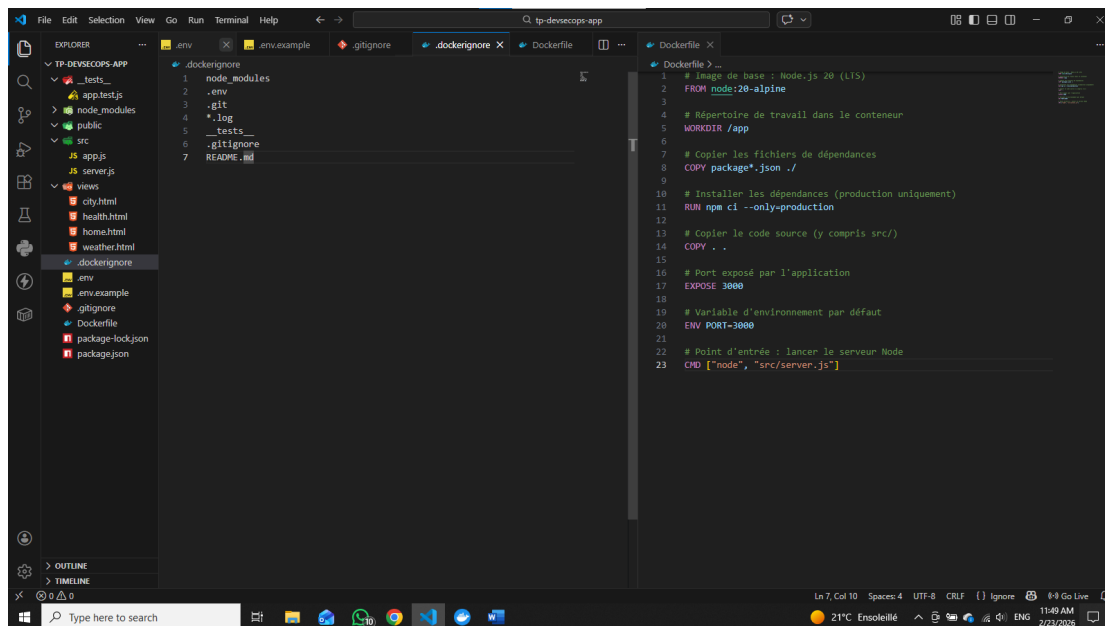


Figure 4.1: Structure du projet dans VS Code – Dockerfile et .dockerignore

```
1 FROM node:20-alpine
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm ci --only=production
5 COPY . .
6 EXPOSE 3000
7 ENV PORT=3000
8 CMD ["node", "src/server.js"]
```

Listing 4.1: Dockerfile – Image de production

Le fichier `.dockerignore` exclut les répertoires non nécessaires à l'image de production (`node_modules`, fichiers de test, artefacts de build) afin de réduire la taille de l'image et d'éviter d'inclure des fichiers sensibles.

4.2 Build et Test Local

Listing 4.2: Commandes Docker – Build local

```
# Build de l'image
docker build -t weatherops:latest .
```

```
# Test local
docker run -p 9000:3000 --env-file .env weatherops:latest

# Vérifier les conteneurs actifs
docker ps
```

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP\Desktop\fourth year\Devsecops\tp-devsecops-app>docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS         NAMES
97512062cb80   tp-devsecops-app:latest             "docker-entrypoint.s..." 2 minutes ago  Up 2 mi
nutes         0.0.0.0:9000->3000/tcp, [::]:9000->3000/tcp   tp-appwheather

C:\Users\HP\Desktop\fourth year\Devsecops\tp-devsecops-app>
```

Figure 4.2: Conteneur Docker WeatherOps en cours d'exécution (port 9000 mappé sur 3000)

4.3 Push vers le Registry GitLab

Listing 4.3: Commandes Docker – Push vers GitLab

```
# Login au registry
docker login registry.gitlab.com

# Tag de l'image
docker tag weatherops:latest $CI_REGISTRY_IMAGE:latest

# Push de l'image
docker push $CI_REGISTRY_IMAGE:latest
```

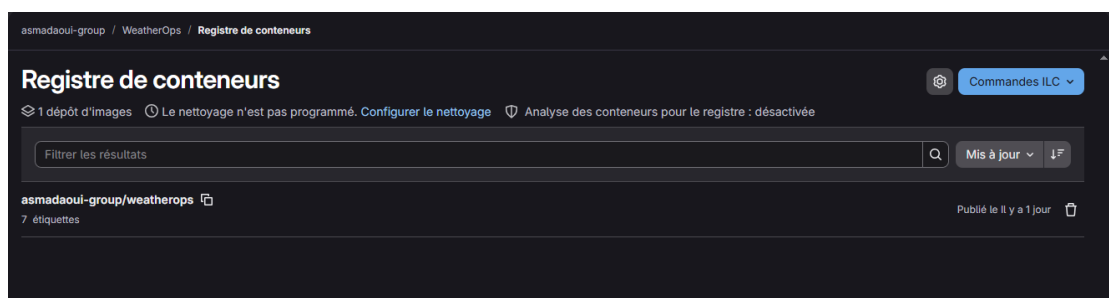


Figure 4.3: Registry GitLab – Image asmadaoui-group/weatherops avec 7 étiquettes publiées

5. Infrastructure AWS et Configuration

5.1 Création de la Paire de Clés SSH

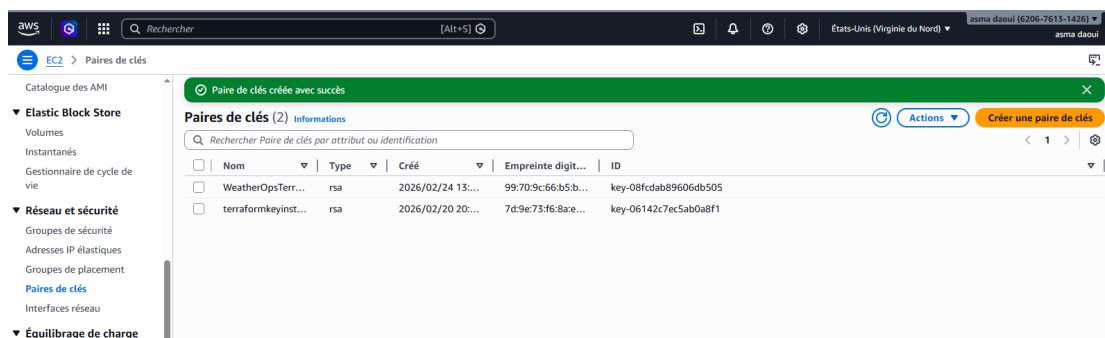


Figure 5.1: Console AWS EC2 – Paire de clés WeatherOpsTerr... créée avec succès pour l'accès SSH

5.2 Instance EC2 de Staging

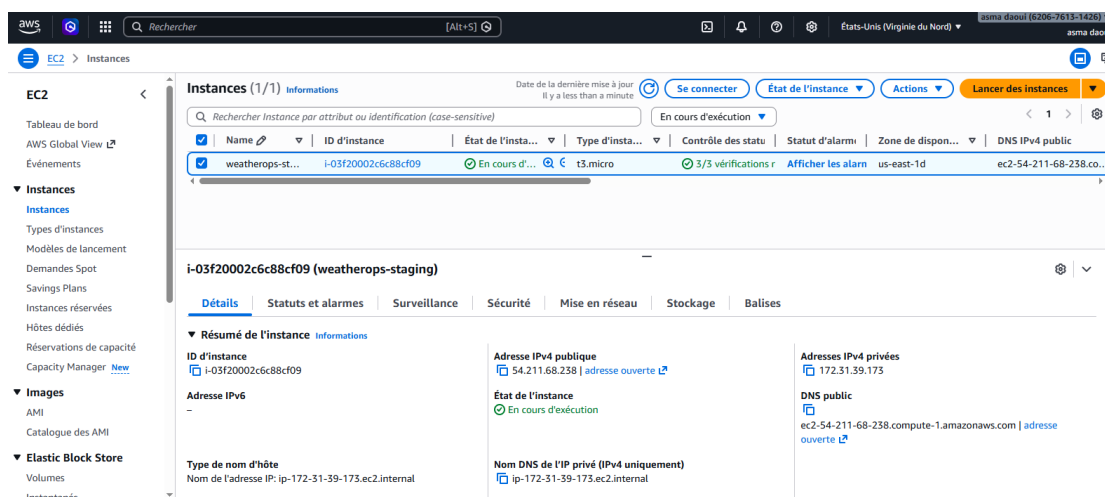


Figure 5.2: Console AWS EC2 – Instance weatherops-staging (t3.micro) en cours d'exécution avec IP publique 54.211.68.238

5.3 Déploiement via Terraform

L'infrastructure a été provisionnée avec Terraform. L'output affiche l'adresse IP publique de l'instance créée.

```
Enter a value: yes
aws_instance.staging: Creating...
aws_instance.staging: Still creating... [00m10s elapsed]
aws_instance.staging: Still creating... [00m20s elapsed]
aws_instance.staging: Still creating... [00m30s elapsed]
aws_instance.staging: Still creating... [00m40s elapsed]
aws_instance.staging: Creation complete after 49s [id=i-03f20002c6c88cf09]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

public_ip = "54.211.68.238"
```

Figure 5.3: Output Terraform – public_ip = "54.211.68.238" – Création complète en 49s

6. Pipeline GitLab CI/CD

6.1 Configuration des Variables CI/CD

Avant l'exécution du pipeline, les variables sensibles ont été configurées dans les paramètres du projet GitLab.

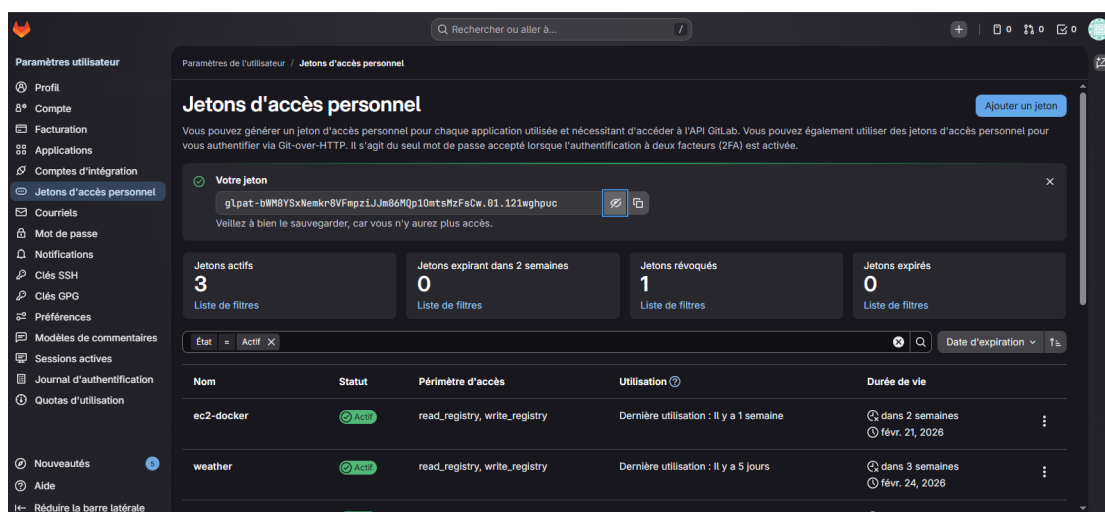


Figure 6.1: GitLab – Jetons d'accès personnels avec périmètre `read_registry`, `write_registry`

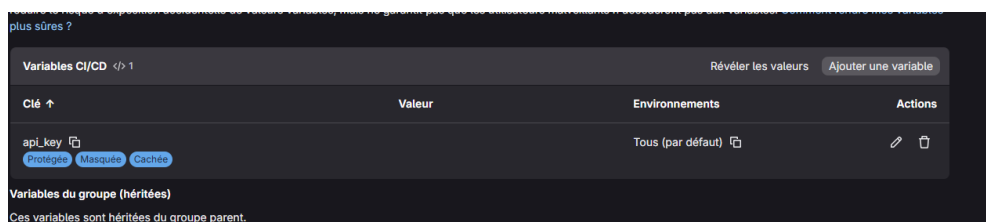
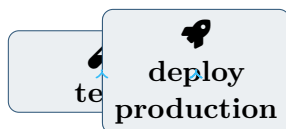


Figure 6.2: GitLab CI/CD Variables – Variable `api_key` configurée (Protégée, Masquée, Cachée)

6.2 Vue d'Ensemble du Pipeline

Le pipeline est défini dans `.gitlab-ci.yml` et comprend 5 stages exécutés séquentiellement :



6.3 Stage 1 – Test

🔧 Stage: test

```
test:
  stage: test
  image: node:20-alpine
  before_script:
    - npm ci
  script:
    - npm test
```

Ce stage installe les dépendances proprement (`npm ci`) et lance la suite de tests Jest. Un échec ici bloque l'ensemble du pipeline.

6.4 Stage 2 – Security (Détection de Secrets)

```
1 security:
2   stage: security
3   image: alpine:latest
4   script:
5     - |
6       if grep -rE 'API_KEY\s*=\s*["\x27]?sk-[a-zA-Z0-9-]+' \
7         --include="*.js" --include="*.json" . 2>/dev/null; then
8         echo "ERREUR : secret detecte"
9         exit 1
10      fi
11      echo "Aucun secret suspect detecte."
```

Listing 6.1: Stage security – Détection de clés API

⚠️ Résultat en Présence du Secret

Avec la valeur `const API_KEY = "sk-demo-123456789"` dans `app.js`, ce stage détecte la fuite et fait échouer le pipeline.

Correction : Utiliser `process.env.API_KEY` et injecter la valeur via les variables CI/CD GitLab.

```

Horodatage du journal en UTC.
22 12:43:49 $ apk add --no-cache grep
23 12:43:49 (1/2) Installing pcre2 (10.47-r0)
24 12:43:49 (2/2) Installing grep (3.12-r0)
25 12:43:49 Executing busybox-1.37.0-r30.trigger
26 12:43:49 OK: 9156 KiB in 18 packages
27 12:43:49 $ echo "🔍 Scanning for potential secrets..." # collapsed multi-line command
28 12:43:49 🔍 Scanning for potential secrets...
29 12:43:49 ./git/logs/refs/remotes/origin/main:0000000000000000000000000000000000000000 fbb15089baec172f7a3628c3
7e9be37dab77a9c root <root@runner-j2nyww-sk-project-79719627-concurrent-0.(none)> 1771937028 +0000
fetch origin --no-recurse-submodules +refs/pipelines/2346276181:refs/pipelines/2346276181 +refs/heads/m
ain:refs/remotes/origin/main --depth 20 --prune --quiet: storing head
30 12:43:49 ./git/logs/HEAD:0000000000000000000000000000000000000000 fbb15089baec172f7a3628c37e9be37dab77a9c root <
root@runner-j2nyww-sk-project-79719627-concurrent-0.(none)> 1771937028 +0000 checkout: moving from n
one to fbb15089baec172f7a3628c37e9be37dab77a9c
31 12:43:49 ./gitlab-ci.yml:# Si le code contient un motif type API_KEY=sk-demo-..., le job échoue
32 12:43:49 ./gitlab-ci.yml: if grep -R -E '(password\s*=\|passwd\s*=\|api[_-]?key\s*=\|secret\s*=\|token\s*=\|sk-
|AKIA|BEGIN PRIVATE KEY)' .; then
33 12:43:49 ./gitlab-ci.yml: - terraform apply -auto-approve -var "docker_image=$CI_REGISTRY_IMAGE" -var "docke
r_tag=$CI_COMMIT_SHORT_SHA" -var "registry_url=$CI_REGISTRY" -var "registry_user=$CI_REGISTRY_USER" -va
r "registry_password=$CI_REGISTRY_PASSWORD"
34 12:43:49 ./gitlab-ci.yml: - terraform apply -auto-approve -var "docker_image=$CI_REGISTRY_IMAGE" -var "docke
r_image_tag=latest" -var "registry_url=https://$CI_REGISTRY" -var "registry_user=$CI_REGISTRY_USER" -va
r "registry_password=$CI_REGISTRY_PASSWORD"
35 12:43:49 ./src/app.js:const apiKey="sk-demo-123456789";
36 12:43:49 ✖ Potential secret detected!
37 12:43:49 Cleaning up project directory and file based variables
38 12:43:50 ERROR: Job failed: exit code 1

```

Figure 6.3: Pipeline échoué – Détection du secret `sk-demo-123456789` dans `src/app.js` (exit code 1)

6.5 Stage 3 – Build (Docker)

```

1 build:
2   stage: build
3   image: docker:24
4   services:
5     - docker:24-dind
6   variables:
7     DOCKER_TLS_CERTDIR: "/certs"
8   script:
9     - docker login -u $CI_REGISTRY_USER \
10       -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
11     - docker build -t $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA \
12       -t $CI_REGISTRY_IMAGE:latest .
13     - docker push $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
14     - docker push $CI_REGISTRY_IMAGE:latest

```

Listing 6.2: Stage build – Build et push de l'image Docker

L'image est taguée avec le SHA court du commit (pour la traçabilité) et `latest` (pour Kubernetes).

6.6 Stage 4 – Deploy Staging

```

1 deploy_staging:
2   stage: deploy_staging
3   image: bitnami/kubectl:latest
4   before_script:
5     - mkdir -p ~/.kube
6     - echo "$KUBE_CONFIG" | base64 -d > ~/.kube/config
7   script:
8     - kubectl apply -f k8s/namespace.yaml
9     - kubectl apply -f k8s/deployment.yaml

```

```

10 - kubectl apply -f k8s/service.yaml
11 - kubectl rollout status deployment/weatherops-app -n
    weatherops
12 - kubectl get pods -n weatherops
13 rules:
14 - if: $CI_COMMIT_BRANCH == "develop"

```

Listing 6.3: Stage deploy_staging – Déploiement automatique sur develop

Ce stage se déclenche **automatiquement** sur la branche `develop`. Il decode la variable CI/CD `$KUBE_CONFIG` (kubeconfig encodé en base64) et applique les manifests Kubernetes.

6.7 Stage 5 – Deploy Production

```

1 deploy_production:
2   stage: deploy_production
3   image: bitnami/kubectl:latest
4   when: manual          # <-- D clenchement manuel obligatoire
5   before_script:
6     - mkdir -p ~/.kube
7     - echo "$KUBE_CONFIG" | base64 -d > ~/.kube/config
8   script:
9     - kubectl apply -f k8s/namespace.yaml
10    - kubectl apply -f k8s/deployment.yaml
11    - kubectl apply -f k8s/service.yaml
12    - kubectl rollout status deployment/weatherops-app -n
        weatherops
13    - kubectl get pods -n weatherops
14   rules:
15    - if: $CI_COMMIT_BRANCH == "main"

```

Listing 6.4: Stage deploy_production – Déploiement manuel sur main

Stratégie de Déploiement

Branche	Comportement
develop	Deploy staging automatique
main	Deploy production manuel (bouton GitLab)

Cette approche garantit qu'aucun déploiement non supervisé n'atteint la production.

6.8 Pipeline Réussi

Après correction du secret hardcodé (remplacement par `process.env.API_KEY`), le pipeline s'exécute avec succès sur toutes les étapes.

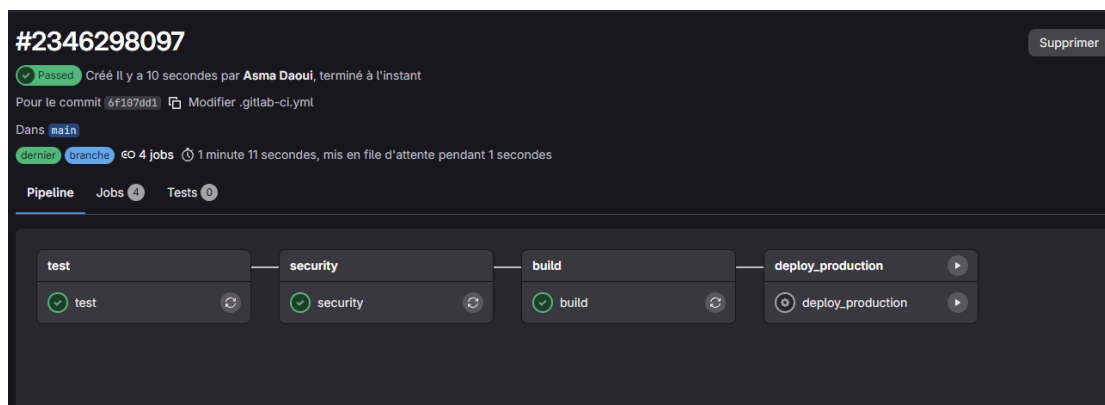


Figure 6.4: Pipeline GitLab CI/CD réussie – 4 jobs passés (test, security, build, deploy_production) en 1min 11s sur la branche main

7. Déploiement Kubernetes

7.1 Structure des Manifests

Les manifests Kubernetes sont organisés dans le répertoire `k8s/` :

- `k8s/namespace.yaml` – Crée le namespace `weatherops`.
- `k8s/deployment.yaml` – Définit le Deployment (pods, replicas, image, env vars).
- `k8s/service.yaml` – Expose le Deployment via un Service (ClusterIP ou Node-Port).

7.2 Vérification du Déploiement

Listing 7.1: Commandes de vérification Kubernetes

```
# Vérifier le statut du rollout
kubectl rollout status deployment/weatherops-app -n weatherops

# Lister les pods
kubectl get pods -n weatherops

# Accéder aux logs d'un pod
kubectl logs -l app=weatherops-app -n weatherops

# Port-forward pour test local
kubectl port-forward svc/weatherops-svc 8080:3000 -n weatherops
```

8. Analyse de Sécurité

8.1 Vulnérabilités Identifiées

Vulnérabilité	Description	Correction
Secret Hardcodé	<code>API_KEY = "sk-demo-..."</code> dans <code>app.js</code>	<code>process.env.API_KEY</code> + CI/CD var
<code>.env</code> en clair	Fichier <code>.env</code> présent dans le repo	Ajouter au <code>.gitignore</code> , utiliser <code>.env.example</code>
Données statiques	Météo simulée sans auth	Acceptable pour TP

8.2 Bonnes Pratiques Appliquées

- `.dockerignore` – exclut `node_modules`, `.env`, tests de l'image.
- `.gitignore` – exclut `.env`, `node_modules`, fichiers sensibles.
- `npm ci` – installation reproductible et déterministe (vs `npm install`).
- `node:20-alpine` – image de base minimale, surface d'attaque réduite.
- `Stage security` – détection de patterns de secrets avant build.

8.3 Améliorations Recommandées

💡 Pistes d'Amélioration

1. Intégrer **SonarQube** ou **Semgrep** pour l'analyse statique (SAST).
2. Ajouter **Trivy** pour le scan de vulnérabilités de l'image Docker.
3. Déployer **OWASP ZAP** en DAST sur l'environnement staging.
4. Utiliser **HashiCorp Vault** ou les secrets GitLab pour la gestion des credentials.
5. Ajouter **NetworkPolicy** Kubernetes pour isoler les pods.

9. Conclusion

Ce TP a permis de mettre en œuvre un pipeline DevSecOps complet autour d'une application Node.js/Express. Les principaux enseignements sont :

- L'intégration de la sécurité dès le début du pipeline (*Shift Left*) via la détection automatique de secrets.
- La valeur des tests automatisés (Jest/Supertest) comme filet de sécurité avant chaque déploiement.
- La séparation des environnements staging/production avec des règles de déclenchement différenciées.
- L'importance du `.gitignore` et du `.dockerignore` pour éviter les fuites de données sensibles.

L'application WeatherOps, bien que simple dans sa logique métier, constitue un socle solide pour illustrer l'ensemble des pratiques DevSecOps attendues en contexte professionnel.

✓ Résumé des Livrables

-  Code source Node.js / Express
-  6 tests Jest/Supertest (100% pass)
Stage de détection de secrets
-  Image Docker multi-tag
-  Déploiement Kubernetes (staging + production)
-  Pipeline GitLab CI/CD 5 stages

A. Variables CI/CD GitLab

Les variables suivantes doivent être configurées dans **Settings > CI/CD > Variables** du projet GitLab :

Variable	Description
CI_REGISTRY_USER	Username pour le registry Docker GitLab
CI_REGISTRY_PASSWORD	Token ou mot de passe registry
CI_REGISTRY	URL du registry (ex. registry.gitlab.com)
CI_REGISTRY_IMAGE	Chemin complet de l'image
KUBE_CONFIG	Kubeconfig encodé en base64

B. Données Météo – Référence

Ville	Pays	Condition	Temp.	Hum.	Vent
Paris	France	Partly Cloudy	18°C	65%	12 km/h
London	United Kingdom	Rainy	14°C	80%	20 km/h
Tokyo	Japan	Sunny	27°C	55%	8 km/h
New York	USA	Clear	22°C	50%	15 km/h
Dubai	UAE	Hot & Sunny	40°C	30%	5 km/h